

## Bahan Ajar Pertemuan 6

---

### BAHAN AJAR - PERTEMUAN 6

---

#### Bubble Sort & Insertion Sort

---

|    |             |
|----|-------------|
| TP | BK 1.1, 1.3 |
|----|-------------|

---

##### A. APA ITU SORTING?

**Sorting (pengurutan)** adalah proses menyusun data dalam urutan tertentu (menaik/ascending atau menurun/descending).

##### Kenapa Sorting Penting?

- Data terurut → binary search bisa digunakan
  - Data terurut → lebih mudah dianalisis
  - Banyak algoritma butuh data terurut
- 

##### B. BUBBLE SORT

###### Prinsip

Bubble sort bekerja dengan **membandingkan dua elemen berdekatan** dan menukarnya jika tidak dalam urutan yang benar. Data terbesar akan “menggelembung” ke posisi akhir seperti gelembung udara.

###### Cara Kerja

|                                           |                   |
|-------------------------------------------|-------------------|
| Array awal: [5, 3, 8, 1]                  |                   |
| Pas 1 (i=0):                              |                   |
| [5, 3, 8, 1] → 5>3 → TUKAR → [3, 5, 8, 1] |                   |
| [3, 5, 8, 1] → 5<8 → TETAP                |                   |
| [3, 5, 8, 1] → 8>1 → TUKAR → [3, 5, 1, 8] | 8 di posisi akhir |
| Pas 2 (i=1):                              |                   |
| [3, 5, 1, 8] → 3<5 → TETAP                |                   |
| [3, 5, 1, 8] → 5>1 → TUKAR → [3, 1, 5, 8] | 5 di posisi akhir |
| Pas 3 (i=2):                              |                   |
| [3, 1, 5, 8] → 3>1 → TUKAR → [1, 3, 5, 8] | URUT!             |

###### Pseudocode

```

PROCEDURE bubbleSort(arr)
  n = length(arr)
  FOR i FROM 0 TO n-2
    FOR j FROM 0 TO n-2-i
      IF arr[j] > arr[j+1] THEN
        TUKAR(arr[j], arr[j+1])
      ENDIF
    ENDFOR
  ENDFOR
END PROCEDURE

```

## Python

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([5, 3, 8, 1])) # [1, 3, 5, 8]

```

## Karakteristik

| Aspek        | Bubble Sort                    |
|--------------|--------------------------------|
| Kompleksitas | $O(n^2)$                       |
| Mudah?       | Sangat mudah dipahami          |
| Cepat?       | Lambat untuk data besar        |
| Stabil?      | Ya (tidak melompati data sama) |

## C. INSERTION SORT

### Prinsip

Insertion sort bekerja seperti **menyusun kartu di tangan**: ambil satu kartu, lalu sisipkan ke posisi yang tepat di antara kartu yang sudah terurut.

### Cara Kerja

Array awal: [5, 3, 8, 1]

Langkah 1: Ambil 5 → [5] (sudah urut)  
 Langkah 2: Ambil 3 → bandingkan dengan 5  
           3 < 5 → sisip di depan → [3, 5]  
 Langkah 3: Ambil 8 → bandingkan dengan 5  
           8 > 5 → sisip di belakang → [3, 5, 8]  
 Langkah 4: Ambil 1 → bandingkan dengan 8, 5, 3  
           1 < 3 → sisip di depan → [1, 3, 5, 8]

## Pseudocode

```
PROCEDURE insertionSort(arr)
  n = length(arr)
  FOR i FROM 1 TO n-1
    key = arr[i]
    j = i - 1
    WHILE j >= 0 AND arr[j] > key
      arr[j + 1] = arr[j]
      j = j - 1
    ENDWHILE
    arr[j + 1] = key
  ENDFOR
END PROCEDURE
```

## Python

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([5, 3, 8, 1])) # [1, 3, 5, 8]
```

## Karakteristik

| Aspek                       | Insertion Sort |
|-----------------------------|----------------|
| Kompleksitas                | $O(n^2)$       |
| Cepat untuk data kecil      | Ya             |
| Cepat untuk data hampirurut | $O(n)$         |
| Stabil?                     | Ya             |

## D. PERBANDINGAN BUBBLE VS INSERTION

| Aspek                 | Bubble Sort         | Insertion Sort           |
|-----------------------|---------------------|--------------------------|
| Cara kerja            | Tukar berpasangan   | Sisipkan ke posisi tepat |
| Mudah dipahami        | ☆☆☆☆☆               | ☆☆☆☆                     |
| Cepat data kecil      | ☆☆                  | ☆☆☆☆                     |
| Cepat data hampirurut | ☆ (tetap $O(n^2)$ ) | ☆☆☆☆ ( $O(n)$ )          |
| Jumlah pertukaran     | Banyak              | Sedikit                  |
| Kompleksitas          | $O(n^2)$            | $O(n^2)$                 |
| Rekomendasi           | Hanya untuk belajar | Lebih baik dari bubble   |

**Visual: Urutkan [2, 4, 6, 8, 1]**

#### **Bubble Sort:**

```
Pas 1: [2,4,6,1,8] – 1 mengapung  
Pas 2: [2,4,1,6,8]  
Pas 3: [2,1,4,6,8]  
Pas 4: [1,2,4,6,8]  
Total: 4 pas
```

#### **Insertion Sort:**

```
Ambil 2 → [2]  
Ambil 4 → [2,4]  
Ambil 6 → [2,4,6]  
Ambil 8 → [2,4,6,8]  
Ambil 1 → [1,2,4,6,8] – geser 4 kali  
Total: 4 langkah (lebih efisien!)
```

---

### **E. RANGKUMAN**

1. **Bubble Sort:** bandingkan & tukar berpasangan → data besar mengapung ke akhir ( $O(n^2)$ )
2. **Insertion Sort:** ambil data → sisipkan ke posisi tepat ( $O(n^2)$ , lebih baik untuk data hampirurut)
3. Keduanya **sederhana** tapi **lambat** untuk data besar
4. Insertion sort **lebih efisien** daripada bubble sort

---

### **MGMP Informatika SMAN 6 Cimahi**